GROUP 4 PROJECT PROPOSAL IPT

ABE Proposal

I. Project Title

ComLease: Commercial Leasing & Billing Management System

II. Introduction

Small commercial building owners who rent out stalls, offices, or stores often manage tenants, lease contracts, and bills manually using notebooks, spreadsheets, or verbal agreements. This can make it difficult to keep track of lease dates, monthly rent, CUSA (Common Use Service Area) fees, utility costs, and payments. As the number of tenants increases, mistakes such as incorrect bills, missed lease renewals, or missing payment records can happen.

The intended users of the system are small commercial building owners/ landlords and the business tenants who rent the units. Owners need an easier way to monitor occupied units, lease contracts, payments, and upcoming renewals. Tenants also need a simple way to check their lease information, monthly bills, and payment status without always having to contact the landlord.

A web-based system can help by keeping all tenant, unit, lease, and billing information in one place. It can help calculate monthly bills, track payments, and remind owners about leases that are about to expire. This can reduce manual work, prevent errors, and make the leasing and billing process easier and more transparent for both owners and tenants.

III. Statement of the Problem

Small commercial building owners often use notebooks or spreadsheets to manage tenants, rental units, lease contracts, and monthly bills. This can cause problems such as wrong records, missed lease renewals, billing errors, and difficulty checking payments and occupied units.

Specific Problem Questions

1. How can tenant and unit information be organized and updated easily?

2. How can lease dates and monthly bills be tracked accurately?
3. How can tenants check their own lease and billing information?
4. How can landlords easily monitor occupied units and payments?

IV. Objectives

General Objective

To develop a web-based system that makes managing commercial leases, tenants, and monthly bills easier and more organized.

Specific Objectives

1. To organize tenant, unit, and lease information in one system.
2. To calculate and track monthly bills and payments accurately.
3. To allow tenants to view their own lease and billing information.
4. To help landlords easily monitor occupied units, payments, and lease status.

V. Target Users

- **Landlord** - Manages the units, tenants, lease contracts, and bills; checks occupied units and payments.
- **Tenants (Business Owners/Lessees)** - Can view their unit, lease status, bills, and payment history.

VI. Proposed Features

- Manage commercial units (add, edit, delete, view)
- Manage tenants/businesses (add, edit, delete, view)
- Manage lease contracts (start date, end date, status)
- Manage billing and payment records (rent + CUSA/utilities, due date, payment status)
- Landlord and Tenant login
- Search tenants, units, and bills
- Filter bills by payment status, and leases by status
- Dashboard showing occupancy and monthly collections

VII. ReactJS Implementation

ReactJS will be used to create the system interface for both the Landlord and Tenant.

- **Components** - Separate parts will be made for the Unit List, Tenant List, Lease Contract, Billing, Payment Form, Login Form, and Dashboard.
- **JSX** - Will be used to display tables and status labels, such as lease and payment status.
- **Props** - will be used to pass unit, tenant, lease, and billing information between components.
- **State** - will be used to manage form inputs, search and filter options, and loading data.
- **Event Handling** - will be used for submitting forms, searching, filtering, and button actions.
- **React Hooks** - useState will manage data, useEffect will load data from the system, and useContext will manage user login information.
- **API Communication** - Axios or Fetch will be used to connect ReactJS to the Laravel backend for managing units, tenants, leases, bills, and payments.

VIII. Laravel Implementation

Laravel will be used for the backend of the system. It will handle the system's data, processes, and connection to the database.

- **Backend Logic** - Will calculate monthly bills, check lease dates, and update payment status.
- **API Endpoints** - RESTful routes such as /api/units, /api/tenants, /api/leases, /api/billing, and /api/payments will be used to connect the frontend and backend. /api/login and /api/logout will be used for user login.
- **Validation** - Laravel Form Requests will check if the information entered, such as lease dates, rent amounts, and tenant details, is complete and correct.
- **Database Operations** - Eloquent ORM will be used to add, view, edit, and delete units, tenants, leases, bills, and payments. It will also connect related data, such as tenants and their units and lease contracts.

- **Authentication** - Laravel Sanctum will be used to manage login and control access based on the user's role, such as Landlord or Tenant.

IX. Database

The main information to be stored includes:

- Commercial unit details such as unit number, floor area, rent, and occupancy status
- Tenant and business information and login details
- Lease contract details such as start date, end date, and renewal status
- Monthly billing details such as rent, CUSA/utility charges, total amount, due date, and payment status
- Payment details such as amount paid, payment date, and payment method

Main tables/entities:

- **users** - (id, name, email, password, role: landlord/tenant)
- **units** - (id, unit_number, floor_area, base_rent, status)
- **tenants** - (id, user_id, business_name, contact_person, contact_info)
- **lease_contracts** - (id, tenant_id, unit_id, start_date, end_date, status)
- **billing_records** - (id, lease_id, month, rent_amount, cusa_amount, total_amount, due_date, status)
- **payments** - (id, billing_record_id, amount_paid, payment_date, method)

X. Cloud Deployment Plan

The React frontend will be hosted using a service such as Vercel or Netlify. The Laravel backend will be hosted using Railway, Render, or a PHP/MySQL hosting service such as Hostinger. The MySQL database will be hosted on the chosen cloud or hosting service. CORS will be set up so the React frontend can connect properly to the Laravel backend. The live system links will be shown during the final presentation.

XI. Expected Output

The expected output is a working web-based Commercial Leasing & Billing Management System (ComLease). The landlord can manage units, tenants, and lease contracts, create and check monthly bills, and monitor payments and occupied units through a dashboard. Tenants can log in to view their lease status, bills, and payment history. The system will also include CRUD operations, search and filter features, and will be available online for the final demonstration.

BERMUDEZ Proposal

I. Project Title

StockTrack: Web-Based Inventory and Stock Management System

II. Introduction

Small businesses, canteens, and school organizations often manage their inventory using handwritten logs or basic spreadsheets. This makes it difficult to track how much stock is on hand, know when items are running low, or see which products move fastest. Errors are common, items get miscounted, stock levels aren't updated in real time, and there is no easy way to search or filter through hundreds of product entries when records grow large.

The intended users of this system are small business owners, store staff, or organization officers who are responsible for managing physical stock, along with admins who oversee overall inventory operations and generate summary reports. These users typically need quick access to accurate stock counts, especially when restocking decisions or sales depend on knowing exactly what is available.

A web based inventory system solves this by centralizing all product and stock data in one place, accessible from any device with internet access. Instead of manually tallying stock on paper, staff can add, update, or remove products directly through a web interface, while the system automatically keeps stock counts accurate and provides tools to search, filter, and monitor inventory levels. This reduces human error, saves time during stock checks, and gives owners/admins clearer visibility into their business operations.

III. Statement of the Problem

Manual inventory tracking methods are time-consuming, error-prone, and lack real-time visibility into current stock levels, making it difficult for business owners and staff to manage products efficiently.

Specific problem questions:

1. How can staff record and update stock levels accurately without relying on manual logs or spreadsheets?
2. How can product information be organized so it can be quickly searched and filtered when the number of items grows large?

3. How can the system alert staff when a product's stock is running low?
4. How can admins get a clear overview of overall inventory status without manually compiling reports?

IV. Objectives

General Objective To design and develop a web-based Inventory and Stock Management System using ReactJS and Laravel that allows businesses to manage and monitor product stock efficiently.

Specific Objectives

1. To develop a system that allows staff to add, update, view, and remove product records (full CRUD).
2. To implement stock quantity tracking that updates automatically when items are added or sold/removed.
3. To provide search and filter functionality so users can quickly locate products by name, category, or stock status.
4. To implement authentication and role-based access for staff and admin accounts.
5. To provide a dashboard summarizing total products, low-stock items, and recent stock activity.
6. To deploy the system online so it can be accessed and used remotely.

V. Target Users

- **Staff/Employees** – add products, update stock levels, and process stock in/out transactions.
- **Admin/Business Owner** – oversees all inventory, manages staff accounts, views reports and dashboard summaries.

VI. Proposed Features

- Authentication with role-based access (staff vs admin)
- Product management with complete CRUD:
    - **Create** – add new products with details (name, category, price, quantity)
    - **Read** – view/browse product list and individual product details
    - **Update** – edit product info and adjust stock quantity (stock in/out)
    - **Delete** – remove discontinued products from inventory
- Category management (add/edit/delete product categories)
- Search products by name or SKU
- Filter products by category or stock status (in stock, low stock, out of stock)
- Low-stock alerts/indicators on the dashboard

- Dashboard showing total products, low-stock count, and recent stock transactions
- Pagination for the product list

VII. ReactJS Implementation

React will be used to build the frontend interface, including:

- **UI Components**: Reusable components for the product table, product form, category list, stock adjustment modal, and dashboard summary cards.
- **Forms**: Controlled form components for adding/editing products (name, category, price, quantity) and for recording stock in/out transactions.
- **State Management**: `useState` for form inputs and local UI state (e.g., modal open/close, selected filters); `useEffect` to fetch product/category data from the API on page load or when filters change.
- **Events**: Event handlers for form submission, search input changes, filter dropdown selection, and button actions (edit, delete, adjust stock).
- **API Communication**: Axios/Fetch calls to Laravel's REST API for authentication, retrieving products, and performing create/update/delete operations, with loading and error states handled in the UI.

VIII. Laravel Implementation

Laravel will handle the backend and API layer, responsible for:

- **Backend Logic**: Automatically recalculating stock quantity when a stock-in or stock-out transaction is recorded, and determining low-stock status based on a threshold.
- **API Endpoints**: RESTful routes (`/api/products`, `/api/categories`, `/api/stock-transactions`, `/api/auth/login`) supporting full CRUD on products and categories.
- **Validation**: Laravel Form Request validation to ensure required fields (product name, valid category, non-negative quantity/price) are correctly submitted before saving.
- **Database Operations**: Eloquent models and migrations to manage products, categories, stock transactions, and users, with relationships (a Product belongs to a Category; a Stock Transaction belongs to a Product and a User).
- **Authentication**: Laravel Sanctum to issue API tokens and restrict certain actions (e.g., deleting products) to admin accounts only.

IX. Database

Main data to be stored:

- User accounts (staff/admin)
- Product categories
- Product details and current stock levels
- Stock transaction history (stock in/out records)

Proposed major tables/entities:

1. **users** – id, name, email, password, role (staff/admin)
2. **categories** – id, name, description
3. **products** – id, name, category_id (FK), sku, price, quantity, low_stock_threshold
4. **stock_transactions** – id, product_id (FK), user_id (FK), type (in/out), quantity, date

X. Cloud Deployment Plan

The React frontend will be deployed on **Vercel** (or Netlify), and the Laravel backend/API will be deployed on **Railway** or **Render**, with a managed MySQL database on the same platform (or a service like Aiven/PlanetScale). Environment variables will store database credentials and the frontend's API base URL, and Laravel's CORS configuration will restrict API access to the deployed frontend domain. Both live links (frontend and backend/API) will be documented and submitted before the final presentation, with the group demonstrating the deployed system live.

XI. Expected Output

A fully functional, deployed web application where staff can log in and manage products through a React interface adding, editing, searching, and removing items while stock levels update automatically and are tracked through a Laravel REST API and MySQL database. Admins can view a dashboard summarizing inventory health, including low-stock alerts, giving the business clear, real-time visibility into its stock without manual record keeping.

ENRIQUEZ Proposal

I. Project Title

OJT Track: A Web-Based On-the-Job Training Hours and Attendance Monitoring System

II. Introduction

College students who undergo On-the-Job Training (OJT) or internship are required to complete a fixed number of training hours before they are allowed to graduate or receive credit for the course. In most schools, this is still tracked using paper Daily Time Records (DTR) that students physically sign and have their company supervisor countersign, then submit to the OJT coordinator for manual tallying. This process is slow, easy to lose or falsify, and makes it difficult for both the student and the coordinator to know at any given moment exactly how many hours have been rendered and how many remain.

The intended users of the proposed system are OJT students, who need a simple way to log their daily time in and time out and track their progress toward their required hours, and OJT coordinators, who need to monitor multiple students at once, verify submitted logs, and quickly identify who is behind schedule. A web-based solution solves this by replacing the paper DTR with a digital log that computes rendered hours automatically, is accessible from any device, and gives the coordinator a real-time overview of every student's progress without manually adding up paper forms.

III. Statement of the Problem

OJT students and coordinators currently have no centralized, digital way to record, verify, and monitor daily training hours, leading to manual tallying errors, delayed verification, and no real-time visibility into a student's progress toward their required OJT hours. Specifically, this study seeks to answer the following questions:

1. How can students record their daily time in and time out and automatically see their total rendered hours instead of manually computing them from paper DTRs?
2. How can OJT coordinators verify and approve submitted time logs and easily identify students who are falling behind on their required hours?
3. How can the system present an accurate, up-to-date view of each student's progress (hours completed vs. hours required) without manual tallying?

IV. Objectives

General Objective

To design and develop a web-based OJT Hours and Attendance Monitoring System using ReactJS and Laravel that allows students to log their daily training hours and allows coordinators to verify logs and monitor student progress through a centralized platform.

Specific Objectives

1. To develop a React-based frontend where students can log daily time in/out entries and view their total rendered hours against their required hours.
2. To develop a Laravel-based backend with API endpoints, validation, and business logic for computing rendered hours and managing log approval.
3. To implement complete CRUD functionality for time log entries and student OJT records, connected end-to-end between React and Laravel.
4. To generate a printable Daily Time Record (DTR) and completion certificate in PDF format based on a student's logged hours.
5. To provide a projected completion date and flag irregular time entries so students and the coordinator can address issues before they affect the student's OJT completion.
6. To deploy the completed system to a cloud platform so that both the frontend and backend are accessible over the internet.

V. Target Users

- OJT Students – log their daily time in/out and monitor their own hours progress.
- OJT Coordinator / Adviser (Admin) – verifies/approves time logs and monitors all students' progress.

VI. Proposed Features

- Student login and daily time in/out logging (with date and computed hours per entry).
- Automatic computation of total rendered hours versus required OJT hours, shown as a simple progress indicator.
- Coordinator dashboard listing all students with their total hours and remaining hours.

- Log approval status (pending, approved, flagged) set by the coordinator.
- Search/filter of time logs or students (e.g., by date range or by status).
- Auto-generated, printable Daily Time Record (DTR) and completion certificate in PDF format, downloadable once a student's required hours are met.
- Projected completion date based on the student's average logged hours per week, so students and the coordinator can see whether the student is on track to finish on time.
- Automatic flagging of irregular time entries (e.g., unusually short sessions or missing time-out) so the coordinator can review them instead of checking every log manually.

VII. ReactJS Implementation

ReactJS will be used to create the system interface for both OJT students and the coordinator.

- **Components —** Separate components will be made for the Time Log Form, Log Table, Progress Bar, Completion Forecast, Flagged Logs List, and Student List.
- **JSX —** Will be used to display tables and status labels, such as log approval status (pending, approved, flagged) and progress toward required hours.
- **Props —** Will be used to pass student, time log, and progress information between components, such as passing a student's required hours into the Progress Bar and Completion Forecast components.
- **State —** Will be used to manage form inputs, search and filter options, and loading data.
- **Event Handling —** Will be used for submitting time log entries, approving or flagging logs, downloading the generated PDF, and searching/filtering.
- **React Hooks —** useState will manage form and filter values, useEffect will load time log and student data from the system, and useContext will manage user login and role information.
- **API Communication —** Axios or Fetch will be used to connect ReactJS to the Laravel backend for submitting time logs, fetching student records, and requesting the DTR/certificate PDF.

VIII. Laravel Implementation

Laravel will be used for the backend of the system. It will handle the system's data, processes, and connection to the database.

- **Backend Logic —** Will compute hours rendered per time log entry and running totals per student, calculate a projected completion date based on average logged hours per week, and automatically flag irregular entries (e.g., unusually short sessions or missing time-out).
- **API Endpoints —** RESTful routes such as /api/logs, /api/students, and /api/dtr (PDF generation) will be used to connect the frontend and backend. /api/login and /api/logout will be used for user login.
- **Validation —** Laravel Form Requests will check that time-out is later than time-in, that required fields are present, and that a log's computed hours are valid before it is saved.
- **Database Operations —** Eloquent ORM will be used to add, view, edit, and delete students and time logs. It will also connect related data, such as a student and their submitted time logs.
- **Authentication —** Laravel Sanctum will be used to manage login and control access based on the user's role, such as Student or Coordinator, restricting log approval and dashboard access to coordinator accounts only.
- **PDF Generation —** A PDF library (such as barryvdh/laravel-dompdf) will be used to generate a formatted Daily Time Record from a student's approved logs, and a completion certificate once rendered hours reach the required total.

IX. Database

The main information to be stored includes:

- Student and account details such as name, login credentials, and role (student/coordinator).
- Each student's OJT details, including their host company, required hours, and total rendered hours.
- Daily time log entries, including time in, time out, computed hours, and approval/flag status.

Main tables/entities:

- users - (id, name, email, password, role: student/coordinator)
- students - (id, user_id, company_name, required_hours, rendered_hours)
- time_logs - (id, student_id, log_date, time_in, time_out, hours, status: pending/approved/flagged)

No additional tables are required for the PDF DTR/certificate generation or the completion forecast, since both are computed on demand from the existing students and time_logs data rather than stored separately.

X. Cloud Deployment Plan

The React frontend will be deployed on Vercel or Netlify, and the Laravel backend together with its MySQL database will be deployed on Railway or Render. The React app will be configured to call the deployed Laravel API through an environment variable for the base API URL, with CORS enabled on Laravel for the frontend's domain. Both live links will be submitted and demonstrated during the final presentation.

XI. Expected Output

The final output is a deployed, full-stack OJT Track web application where students can log their daily time in/out and see their rendered hours, projected completion date, and flagged entries update automatically, and where the OJT coordinator can review, approve, and monitor every student's progress from a single dashboard. Students will be able to download an auto-generated, printable DTR at any time and a completion certificate once their required hours are met. The system will demonstrate complete CRUD for student and time log records, role-based access between students and the coordinator, search/filter of logs, PDF document generation, computed completion forecasting, automatic anomaly flagging, and a live cloud deployment ready for demonstration.

GUINTO Proposal

I. Project Title

Integrated Employee Management & HR Systems

II. Introduction

Small to medium-sized organizations often rely on manual systems or separated spreadsheets to store data about their employee's information, attendance, and leave requests. As a result of this situation, the businesses can encounter issues related to the overall effectiveness of their systems, which can lead to problems with data accuracy and delays in payroll processes. It is easy to see that it is important to set up the system that would store information about employees and would allow them to automate processes in the HR field.

The system will have three categories of users: HR Administrators who manage documents related to employees and set up the system, Department Managers who approve applications for leaves and manage attendance of employees, and Employees who apply for leaves and can see their documents or payslips.

Some businesses, which do not have properly organized data suffer from losses in productivity as well as compliance issues. Delays in the process of approvals of leaves are usual, as well as lower accuracy of attendance recording and with the processing of payroll. The implementation of the digital human resource system can help save money, improve accuracy of data and provide visibility into workforce metrics.

A web-based platform provides 24/7 access through any device, consolidates employee information into a secure database, automates the workflow approval process, provides real-time reports and eliminates the need for any paper-based tasks.

III. Statement of the Problem

Currently, there exist no integrated systems for business organizations for managing their employee information, attendance, level and compensation. Therefore, numerous questions can arise from such a situation as:

1. How is it possible to ensure protection for every one of the employee's files?
2. What can be done to increase accuracy of attendance management?
3. How can HR functions be made accessible to managers and employees without extra inputs?
4. What measures can be taken by human resources for automating leave approval and ensuring timely processing?
5. In what ways can it be possible to enhance employee payroll computations?

IV. Objectives

To develop a fully integrated Employee Management & HR System that facilitates employee data management, automates HR processes (leave approvals, attendance tracking) and optimizes payroll system by means of React and Laravel technologies.

Specific Objectives:

1. To design a complete and safe database to store information regarding employees, including their data, department, position, contact information, and employment status.
2. To create an automated attendance monitoring system that records the time of arrival and leaving of employees, calculates their working hours and prepares reports on attendance for payroll.
3. To design a leave approval system where employees can submit their leave requests and managers can either approve or deny them as well as keep track of the current number of leaves available for each employee.
4. To create a payroll system that calculates employee’s salaries according to their attendance records.
5. To develop HR dashboards and reports that include information about the number of employees, their attendance and leave utilization.
6. To develop different levels of access (Admin, Manager, Employee)

V. Target Users

- **HR Administrators** – Maintaining employee's database (add, update, delete records), setting leave categories, remuneration rates and deductions, processing and finalizing payroll, and to generate compliance reports, and handling system users' access and rights.

- **Department Heads** – Accessing information about the attendance and leave status of employees, approving or declining requests for leave, check leave balance for departments, and get a payroll summaries for department heads.

- **Employees** – Review their records and attendance reports, apply for leave (by stating reasons and dates), determine their leave balance, retrieve payslips and employment certificates, and update their personal information (contact information, emergency contacts)

VI. Proposed Features

Module	Description	Functions
Employee Management	Centralized system for storing, organizing, and managing complete employee information throughout their employment lifecycle.	● Add, edit, delete employee records ● Store personal and professional details ● Track employment status (active, on leave, resigned) ● Manage emergency contacts ● Store employee documents (contracts, certificates)
Attendance Management	Digital time tracking system that records employee work hours, calculates attendance metrics, and generates attendance reports for payroll and compliance.	● Check-in / Check-out recording ● Automatic working hours calculation ● Attendance Reports ● Real-time attendance dashboard ● Absent/late tracking and alerts
Leave Management	Comprehensive leave workflow system that allows employees to request time off, managers to approve/reject requests, and the system to track leave balance	● Configure leave types (annual, sick, casual, etc.) ● Set organizational leave policies ● Employee leave request

	and policies.	submission ● Manager approval/rejection workflow ● Automation leave balance tracking ● Team availability calendar ● Email notifications
Payroll Module	Automated payroll processing system that calculates salaries based on attendance, applies deductions, and generates payslips with complete audit trails.	● Salary structure configuration (base pay, allowances, deductions) ● Automatic salary calculation based on attendance ● Tax and statutory dedication management ● PDF payslip generation ● Payroll history and audit
Reporting & Analytics	Comprehensive reporting and analytics platform providing HR insights through dashboards, custom reports, and trend analysis across all HR functions.	● HR metrics dashboard (employee count, leave utilization, attendance trends) ● Attendance, leave utilization, and payroll reports ● Employee turnover analytics ● Department-level analysis ● Custom report builder for specific data requirements
User & Access Management	Security and authorization system that controls who can access what information based on their role, with complete activity tracking for compliance and auditing.	● Role-based access control ● Secure authentication with JWT tokens ● User authorization and permission verification ● Complete activity logging and audit trails ● Secure password management and encryption
Applicant Tracking System (ATS)	Recruitment management platform that tracks job applicants through the entire hiring pipeline from application to hiring, with document storage and interview management	● Job posting creation and management ● Applicant profile and status tracking ● Resume, 201 files, and certificate storage ● Interview scheduling and conflict prevention

		• Interview feedback and scoring • Applicant history and communication notes • Recruitment pipeline analytics
Policy Management & Distribution	Enterprise document management system for creating, approving, and disseminating company policies with version control and employee acknowledgement tracking for compliance.	• Policy creation with version control • Milti-storage approval workflow • Role based policy distribution and access • Employee policy acknowledgement tracking • Policy search and categorization • Expirations alerts for time-sensitive policies • Complete audit trails of access • Bulk email dissemination

VII. ReactJS Implementation

ReactJS will be used to create the user interface for the HR system, providing interactive dashboards, forms, and real-time data management for employees, managers, and administrators.

- **Components** – Reusable UI components will be created for navigation, dashboards, employee lists, leave requests, attendance tracking, applicant profiles, interview feedback, and policy viewing.
- **Forms** – Input forms for employee registration, leave requests, payroll configuration, applicant uploads (resume, 201 file, certificates), interview feedback, and policy creation/approval.
- **State Management** – Context API or Redux will manage global state for user authentication, employee data, leave requests, attendance records, applicants, policies, and notifications.
- **Props** – Component data will be passed through props for displaying employee information, leave details, applicant documents, and policy content across different pages.
- **Event Handling** – Form submissions, button clicks (edit, delete, approve, reject), table interactions, and modal triggers will handle user actions and send data to the backend.

- **React Hooks** – useState for form inputs and UI state, useEffect for loading data from the backend, useContext for authentication and global data management.
- **API Communication** – Fetch or Axios will connect ReactJS to the Laravel backend for CRUD operations on employees, leaves, attendance, payroll, applicants, interviews, and policies.

VIII. Laravel Implementation

Laravel will be used to build the backend system that handles data processing, business logic, database operations, and API endpoints for managing employees, attendance, leave, payroll, applicants, and policies.

- **Backend Logic** – Process employee registrations, calculate leave balances, compute working hours, calculate payroll with deductions, validate applicant documents and interview workflows, manage policy approvals and disseminations, and send automated notifications.
- **API Endpoints** – RESTful routes such as /api/employees, /api/leave-requests, /api/attendance, /api/payroll, /api/applicants, /api/interviews, and /api/policies will connect the React frontend to the backend for all CRUD operations and approvals.
- **Validation** – Laravel Form Requests will validate employee details (email uniqueness, phone format), leave dates (no overlaps, sufficient balance), applicant documents (file type and size), interview scheduling (no conflicts), and policy workflow state transitions.
- **Database Operations** – Eloquent ORM will handle adding, viewing, editing, and deleting records for employees, leaves, attendance, payroll, applicants, interviews, and policies. Relationships will connect related data (employees to departments, applicants to documents, policies to versions).

IX. Database

The main information to be stored includes:

- Employee details such as name, ID, department, designation, contact information, and employment status
- Attendance records including check-in/check-out times, working hours, and daily status
- Leave requests with types, dates, approval status, and leave balances
- Payroll information such as salary structure, deductions, and payslips
- Applicant profiles with job applications, status tracking, and uploaded documents (resume, 201 file, certificates)
- Interview records with feedback, scores, and hiring decisions

- Company policies with versions, approval workflows, and employee acknowledgment tracking

Main tables/entities:

- **users** – (user_id, email, password, role: admin/manager/employee)
- **employees** – (employee_id, first_name, last_name, phone, email, department_id, designation, employment_status)
- **departments** – (department_id, department_name, manager_id)
- **attendance** – (attendance_id, employee_id, attendance_date, check_in_time, check_out_time, working_hours, status)
- **leave_types** – (leave_type_id, leave_type_name, max_days_per_year)
- **leave_requests** – (leave_request_id, employee_id, leave_type_id, start_date, end_date, status, approver_id)
- **leave_balance** – (leave_balance_id, employee_id, leave_type_id, total_days, used_days, remaining_days)
- **salary_structure** – (salary_structure_id, employee_id, basic_salary, allowances, effective_date)
- **payslips** – (payslip_id, employee_id, payroll_month, gross_salary, total_deductions, net_salary)
- **job_postings** – (job_posting_id, job_title, department_id, posted_date, closing_date, status)
- **applicants** – (applicant_id, job_posting_id, first_name, email, phone, application_status, application_date)
- **applicant_documents** – (document_id, applicant_id, document_type, file_name, file_path)
- **interviews** – (interview_id, applicant_id, interview_date, interviewer_id, interview_type, status)
- **interview_feedback** – (feedback_id, interview_id, technical_score, communication_score, recommendation)
- **policies** – (policy_id, policy_title, policy_category, policy_status, created_by, effective_date)
- **policy_versions** – (version_id, policy_id, version_number, content, approval_status)
- **policy_acknowledgments** – (acknowledgment_id, policy_id, employee_id, acknowledged_date)

X. Cloud Deployment Plan

The React frontend will be hosted using services such as Vercel or Netlify with automatic deployments from GitHub. The Laravel backend will be hosted on platforms like Railway, Render, or traditional PHP/MySQL hosting services such as Hostinger or

AWS. The MySQL database will be hosted on a managed database service (AWS RDS or DigitalOcean Managed Database) for reliability and automatic backups. CORS will be configured to allow secure communication between the React frontend and Laravel backend. API endpoints will be secured with JWT authentication.

XI. Expected Output

The expected output is a working, production-ready web-based Employee Management & HR System (HR-Integrated). The system will enable HR administrators and managers to efficiently manage employee records, track attendance, process leave requests, handle payroll operations, manage job applicants and interviews, and distribute company policies with compliance tracking. Employees can view their personal records, submit leave requests, check attendance and payroll information, view and acknowledge company policies, and apply for job openings. The system includes a responsive dashboard for all user roles, role-based access control, real-time notifications, comprehensive reporting and analytics, secure authentication, and automated workflows for approvals. The complete frontend, backend API, database, and deployment links will be presented during the final demonstration.

TANIA Proposal

I. Project Title

OrgConnect: A Web-Based Platform for Student Organization Management, Event Coordination, and Peer Tutoring

II. Introduction

Student organizations coordinate events, membership records, volunteer hours, and peer tutoring largely through spreadsheets, group chats, and social media posts. This leads to buried announcements, manual attendance tracking, and inconsistent volunteer-hour records.

The intended users are organization officers, who need to publish events and verify participation, general members, who need one place to register for events and track their own involvement and underclassmen, who benefit from visibility into opportunities and access to peer tutoring.

A centralized system is needed because the manual approach does not scale as membership grows, making it hard to track attendance, verify volunteer hours, or generate reports. A web-based solution solves this by giving officers and members real-time, accessible records through a single React and Laravel application.

III. Statement of the Problem

Student organizations lack a centralized digital platform to manage events, track membership and volunteer hours, and facilitate peer tutoring and resource-sharing among members. This results in inefficient coordination, inconsistent records, and reduced visibility of opportunities for members and underclassmen.

Specific Problem Questions

1. How can event and opportunity postings be created, published, and made easily accessible and searchable for all members?
2. How can membership records and volunteer hours be accurately logged, verified, and reported without relying on manual spreadsheets?
3. How can peer tutoring sessions and learning resources be organized, requested, and tracked in a structured and searchable manner?
4. How can officers gain a clear, real-time view of organizational activity (attendance, hours, participation) to support decision-making and recognition?

IV. Objectives

General Objective

To design and develop OrgConnect, a full-stack web application using ReactJS and Laravel that centralizes event management, membership and volunteer hour tracking, and peer tutoring for student organizations.

Specific Objectives

1. To develop a ReactJS frontend that allows officers and members to create, view, update, and manage events, membership records, and tutoring sessions through an intuitive interface.
2. To build a Laravel backend with RESTful API endpoints, input validation, and role-based business logic that supports complete CRUD operations across all modules.
3. To design and implement a relational database that accurately stores and relates organizational data, including events, members, volunteer hours, and tutoring records.
4. To deploy the completed application to a cloud platform so that both the frontend and backend are accessible online for live use and demonstration.

V. Target Users

- **Organization Officers / Administrators:** create and manage events, review membership applications, approve volunteer hours and tutoring requests, and view analytics.
- **General Members:** browse and register for events, log and track their own volunteer hours, request tutoring sessions, and receive notifications and announcements.
- **Underclassmen / Prospective Members:** discover opportunities, browse shared learning resources, and request peer tutoring sessions.
- **Peer Tutors:** members who post tutoring sessions or resources and manage incoming session requests.

VI. Proposed Features

The system has three core modules: Event & Opportunity Management, Membership & Volunteer Hours Tracking, and Peer Tutoring & Resource Sharing plus platform-wide features, listed below.

MODULE	FEATURE	TYPE
Event & Opportunity Management	Create, edit, delete, and view events/opportunities	Core CRUD
Event & Opportunity Management	Search and filter events by keyword and category	Baseline additional
Event & Opportunity Management	Event categories / tags (workshop, seminar, community service, social)	Additional feature
Event & Opportunity Management	RSVP and attendance tracking	Core function
Membership & Volunteer Hours	Add, edit, and remove member records; log volunteer hours	Core CRUD
Membership & Volunteer Hours	Authentication with role-based access (officer vs. member)	Baseline additional
Membership & Volunteer Hours	Reports / exportable summary of hours per member	Baseline additional
Peer Tutoring & Resource Sharing	Post, edit, and delete tutoring sessions and resources	Core CRUD
Peer Tutoring & Resource Sharing	Status management (pending, confirmed, completed)	Baseline additional
Peer Tutoring & Resource Sharing	Application / request approval flow (request → officer approval → status update)	Additional feature
Platform-wide	Officer/admin dashboard with analytics (members, events, hours logged, active members)	Additional feature
Platform-wide	Announcements / notice board	Additional feature
Platform-wide	Certificate / attendance record generator (auto-generated after event completion)	Additional feature

Platform-wide	In-app notifications (e.g., “your hours were approved”, “new event posted”)	Additional feature
Platform-wide	Leaderboard / recognition based on volunteer hours and participation	Additional feature
Platform-wide	Pagination on all list views (events, members, sessions)	Baseline additional

VII. ReactJS Implementation

ReactJS will be used to build the interface for both Officers and Members.

- **Components:** Separate components will be made for the Event List, Member List, Volunteer Hours, Tutoring Sessions, Announcements, Login Form, and Dashboard.
- **JSX:** Will be used to display tables and status labels, such as event RSVP status, hour approval status, and tutoring request status.
- **Props:** Will be used to pass event, member, volunteer hour, and tutoring session information between components.
- **State:** Will be used to manage form inputs, search and filter options, and loading data.
- **Event Handling:** Will be used for submitting forms, RSVPs, approving/rejecting requests, searching, and filtering.
- **React Hooks:** useState will manage data, useEffect will load data from the system, and useContext will manage user login and role information.
- **API Communication:** Axios or Fetch will be used to connect ReactJS to the Laravel backend for managing events, members, volunteer hours, tutoring sessions, and announcements.

VIII. Laravel Implementation

Laravel will be used for the backend of the system. It will handle the system's data, processes, and connection to the database.

- **Backend Logic:** Will calculate total volunteer hours, update event attendance and RSVP status, and process tutoring request approvals.
- **API Endpoints:** RESTful routes such as /api/events, /api/members, /api/hours, /api/tutoring-sessions, and /api/announcements will connect the frontend and backend. /api/login and /api/logout will be used for user login.
- **Validation:** Laravel Form Requests will check if the information entered, such as event dates, hour logs, and tutoring session details, is complete and correct.
- **Database Operations:** Eloquent ORM will be used to add, view, edit, and delete events, members, volunteer hours, and tutoring sessions. It will also connect

related data, such as members and their hour logs, or tutoring sessions and their requests.

- **Authentication:** Laravel Sanctum will be used to manage login and control access based on the user's role, such as Officer or Member.

IX. Database

The system will use MySQL as its relational database. The main data stored includes user accounts and roles, event and opportunity postings, event registrations and attendance, membership records, volunteer hour logs, tutoring sessions and requests, shared resources, announcements, notifications, and generated certificates. The proposed major tables are outlined below.

ENTITY / TABLE	KEY FIELDS	PURPOSE
users	id, name, email, password, role (officer/member), created_at	Stores all system accounts and their access role
events	id, title, description, category, date, location, created_by	Stores event and opportunity postings
event_registrations	id, event_id, user_id, status (registered/attended)	Tracks RSVPs and attendance per event
membership_records	id, user_id, join_date, status, position	Stores official membership information
volunteer_hours	id, user_id, activity, hours, date, status (pending/approved)	Logs and verifies volunteer hours
tutoring_sessions	id, tutor_id, subject, description, schedule, status	Stores peer tutoring session postings
tutoring_requests	id, session_id, requester_id, status (pending/approved/rejected)	Handles the approval flow for session requests
resources	id, title, file_path/link, subject, uploaded_by	Stores shared knowledge/resource links
announcements	id, title, content, posted_by, created_at	Stores org-wide notice board posts
notifications	id, user_id, message, is_read, created_at	Stores in-app notification records

certificates	id, user_id, event_id, issued_at, file_path	Stores generated attendance/certificate records

X. Cloud Deployment Plan

The React frontend will be deployed to Vercel or Netlify, and the Laravel backend/API to a PHP-compatible host such as Railway or Render, with MySQL hosted via a managed database service. CORS and environment variables will be configured so the deployed frontend and backend communicate correctly, and both links will be submitted during the final presentation.

XI. Expected Output

The final output is OrgConnect, a deployed full-stack web application where officers manage events, membership, and volunteer hours, and members/underclassmen discover events, log participation, and request peer tutoring through a React frontend, a Laravel REST API, and a MySQL database, with all features in Section VI implemented and demonstrated live.